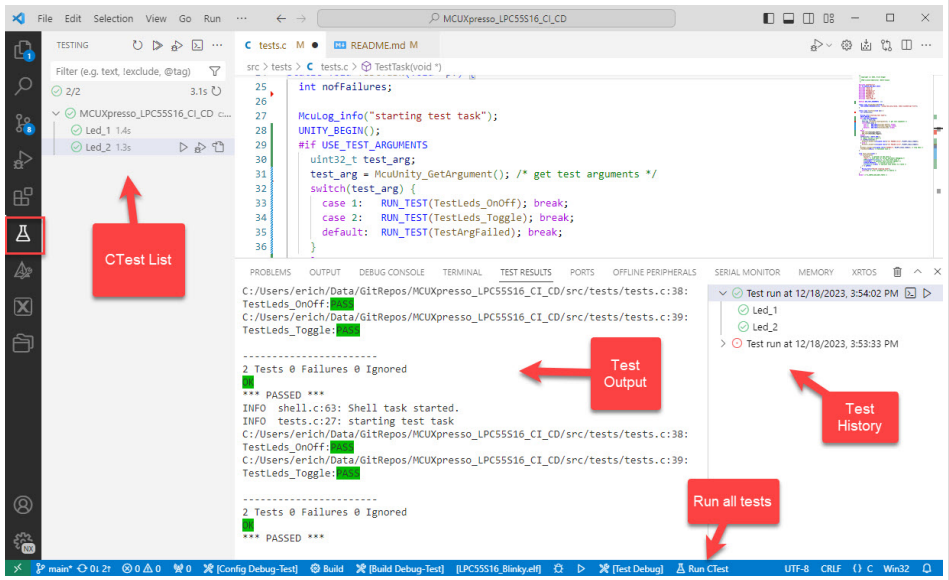
Modern On-Target Embedded System Testing with CMake and CTest

Posted on December 18, 2023 by Erich Styger

One key element of a CI/CD pipeline is the automatic testing phase: whenever I check in new source code or manually trigger it, I can run a test suite to make sure that the changes do not break anything. For this, I have to run automated tests. For an an embedded target, it means that I have to run some tests on the board itself too.

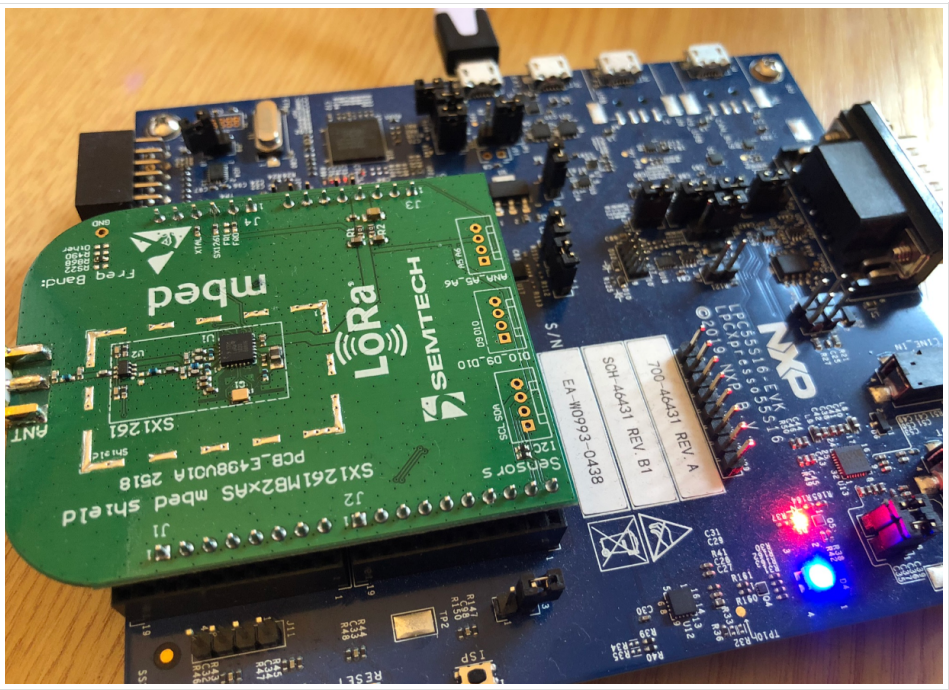


— CTest with VS Code

Outline

In a TDD (Test Driven Development) world you would first write the tests, then develop the software. That would be ideal, but as we all know: the world is not ideal. Instead, in many cases you might have to work on legacy code, and adding tests later on. In this article I describe, how tests can be added to a CMake based project, using the Unity test framework and **CTest as testing driver**.

For example: in a embedded project where LPC55S16 is used to communicate over LoRaWAN, we need to run a lot of tests on the actual hardware:



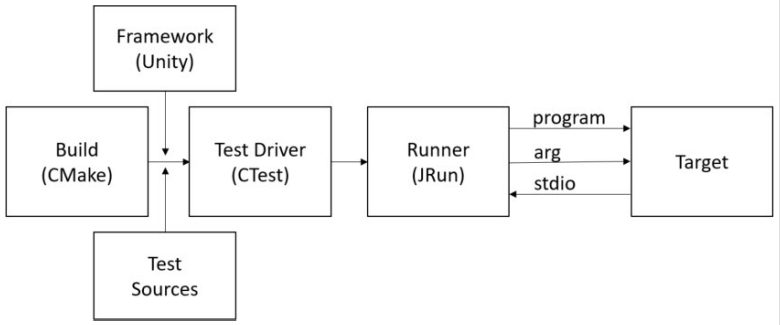
— LoRaWAN with LPC55S16

CI/CD (Continuous Integration / Continuous Delivery) means running lots of tests, and fast. Ideally you would run all the tests on the host. But certain tests needs to run on the hardware and can not run on the host or in a simulator or emulator. Here I show how to run tests **on-target** with the help of a debug probe.

You can find an example project using the approach presented here on GitHub. I recommend that you have a read at my previous topic here: CI/CD for Embedded with VS Code, Docker and GitHub Actions.

In this article I'm using VS Code with the NXP LPC55S16-EVK, but the concepts can be easily used for any other environment:

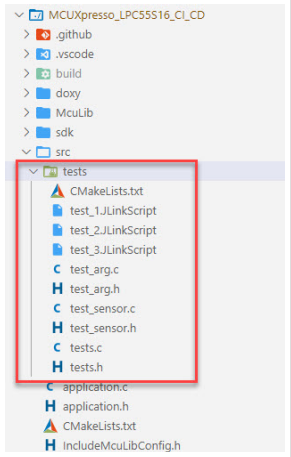
- How to structure test files with CMake and Presets
- Integrating the Unity test framework
- Writing tests with Unity
- Adding tests with CTest
- Configuring test properties with CTest
- Running Tests with JRun to program it, passing arguments and producing output with RTT
- Passing arguments to the application and producing test output
- Executing tests with CTest
- Configuring CTest with regular expressions for pass and fail
- Running tests with Visual Studio Code



— Test Pipeline

Structuring Tests

There are many different ways to organize the test files. My preference is to have them (Unit Tests, System Tests, …) are organized in separate sub-folders, inside the project sources:



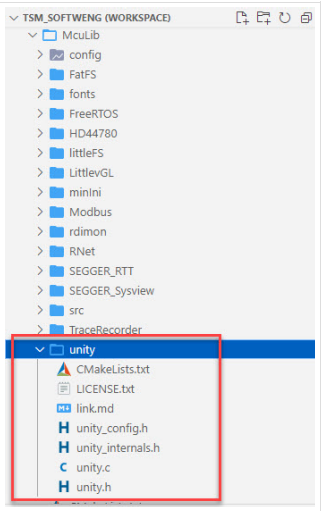
— Tests Folder

So for example the test files for the '`src`' folder are located in '`src/tests`', and so on. Each tests folder has its own `CMakeLists.txt` which is only used if I'm building for testing.

Unity Testing Framework

As testing framework, I'm using Unity from ThrowTheSwitch. This is a very easy-to-use and powerful unit testing framework, but easily can be used for any other testing like integration or system level testing. The Unity test framework has been recently added to the McuLib library of embedded software.

The framework only has a few source files. Unity.h is the header file to be included in the test sources, and unity.c contains the framework to ramp-up and ramp-down the tests:



— Unity Test Framework

Similar to other frameworks, Unity provids a rich set of assert macros. I can write with asserts like this:

```
void TestLeds_OnOff(void) {
  Leds_On(LEDS_BLUE);
  TEST_ASSERT_MESSAGE(Leds_Get(LEDS_BLUE), "Blue LED shall be on");
  Leds_Off(LEDS_BLUE);
  TEST_ASSERT_MESSAGE(!Leds_Get(LEDS_BLUE), "Blue LED shall be off");
}
```

For a full list of possible assertions see https://github.com/ThrowTheSwitch/Unity/blob/master/docs/UnityAssertionsReference.md.

And then run multiple tests like this:

```
UNITY_BEGIN();
RUN_TEST(TestLeds_OnOff);
RUN_TEST(TestLeds_Toggle);
nofFailures = UNITY_END();
```

The UNITY_BEGIN() starts the testing (initializes status, etc), then I run the tests with RUN_TEST() and finally UNITY_END() returns the number of failed tests.

Unity does not have any hardware dependency, so I need to configure it how it shall write the output messages. The framework is configured by the **McuUnity** module:

```
#include "McuUnity.h"
#define UNITY_OUTPUT_CHAR(a)                    McuUnity_putc(a)
#define UNITY_OUTPUT_FLUSH()                    McuUnity_flush()
#define UNITY_OUTPUT_START()                    McuUnity_start()
#define UNITY_OUTPUT_COMPLETE()                 McuUnity_complete()
#define UNITY_OUTPUT_COLOR                      /* use colored output */
```

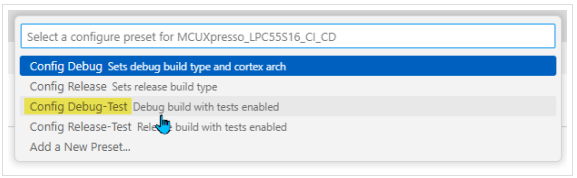
For example it is configured to send all the output to the SEGGER RTT communication channel in McuUnity.c:

```
void McuUnity_putc(int c) {
  McuRTT_StdIOSendChar(c); /* using JRun with RTT */
}
```

I could use other ways than RTT, for example writing to a serial port or using semihosting.

CMake Presets

I'm using a dedicated CMake Preset for testing (see Building with CMake Presets):



— CMake Test Preset

So I have configurations both for the 'debug' and 'release'. For some systems it might make sense only to test the release build, but having a debug build makes it easier to locate testing errors or any other kind of issues.

The 'test' presets are setting a CMake variable **ENABLE_UNIT_TESTING**:

```
{
  "name": "debug-test",
  "displayName": "Config Debug-Test",
  "description": "Debug build with tests enabled",
  "inherits": "debug",
  "cacheVariables": {
    "ENABLE_UNIT_TESTING": "ON"
  }
},
```

That variable is used in the CMakeLists.txt to turn on or off testing support.

Build with Testing Support

The CMake variable **ENABLE_UNIT_TESTING** is then used in the main CMakeLists.txt to set a compiler define:

```
# Check if ENABLE_UNIT_TESTING is set to ON in the CMake preset
if (ENABLE_UNIT_TESTING)
  message(STATUS "ENABLE_UNIT_TESTING is ON")
  add_compile_options(-DENABLE_UNIT_TESTS=1) # used to enable tests in code
else()
  message(STATUS "ENABLE_UNIT_TESTING is OFF")
  add_compile_options(-DENABLE_UNIT_TESTS=0) # used to disable tests in code
endif()
```

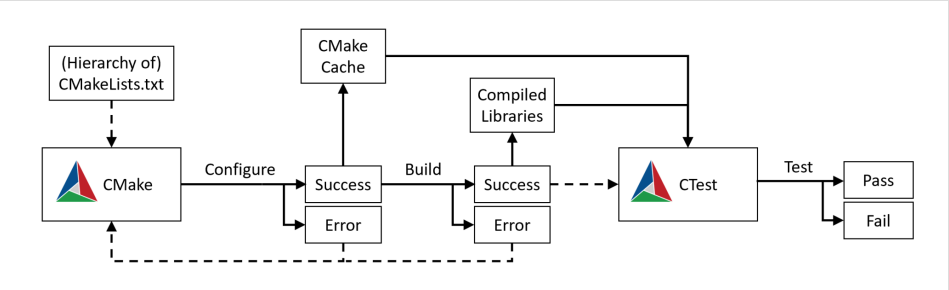
The #define is used in the application to check if it has to build and include the test functions:

```
#define PL_CONFIG_USE_UNIT_TESTS        (1 && defined(ENABLE_UNIT_TESTS) && ENABLE_UNIT_TESTS==1) /* if using unit tests. ENABLE_UNIT_TESTS is set by tl
```

```
#if PL_CONFIG_USE_UNIT_TESTS
  Tests_Init();
#endif
```

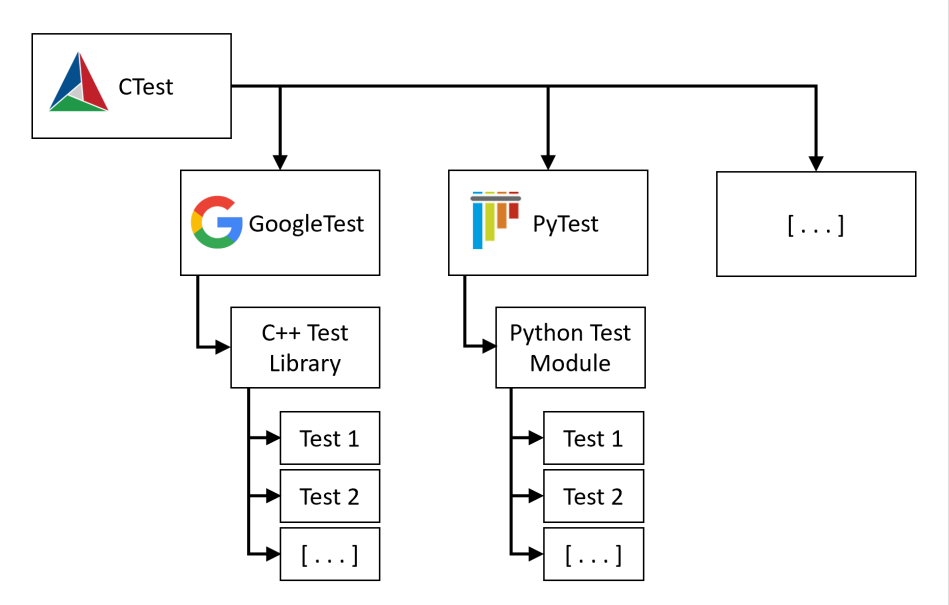
CTest and CMake

CTest is part of the CMake software suite of tools: with added CTest support to the CMakeLists.txt, I can run tests at the end of a successful build:



— CMake with CTest (Source: o3de.org)

CTest is a driver to kick off a test runner like Google Test or PyTest, or anything you like to run:



— CTest (Source: o3de.org)

CTest gets enabled for CMake based projects by including the following in the `CMakeLists.txt`:

```
include(CTest)
```

I prefer to have verbose output, and I can do this with setting the CMake CTest arguments:

```
list(APPEND CMAKE_CTEST_ARGUMENTS "--output-on-failure")
list(APPEND CMAKE_CTEST_ARGUMENTS "--verbose")
```

Finally I need to make sure that I include the Unity framework, as used by my tests:

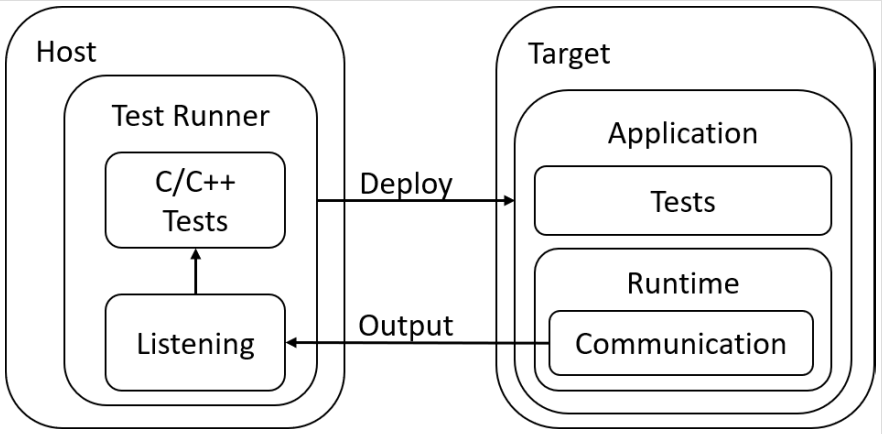
```
add_subdirectory(${MCULIB_DIR}/unity   unity)
```

In a similar way I reference the tests for the src folder from its CMakeLists.txt and link with it:

```
if (ENABLE_UNIT_TESTING)
  message(STATUS "Adding src tests")
  add_subdirectory(./tests                  tests)
endif()
...
if (ENABLE_UNIT_TESTING)
  message(STATUS "Adding src tests library")
  target_link_libraries(
    ${THIS_LIBRARY_NAME}
    PUBLIC srcTestsLib
  )
endif()
```

JRun

To test the binary on the target, I have to deploy it, run it and analyze the output:



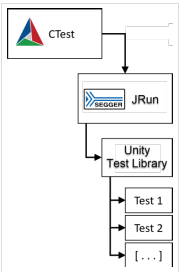
Running that on the host is easy: just run the executable:

```
$ .\testApp --test=1
```

On an embedded target things are a bit more complex, as I have to program the binary for example with a debugger or boot loader.

And unlike for running an executable on the host, I cannot easily pass arguments to the application under test (`--test=1`). Moreover, there is typically no standard input or output, so the application cannot simply `printf()` the test results.

A solution to this problem is using a debugger with Semihosting or RTT. But there is even a simpler way: SEGGER JRun: In this project I'm using the SEGGER JRun with the Unity testing library. JRun is part of the J-Link software suite.



— JRun with Unity and CTest

JRun is a command-line utility which can program a file like a debugger, but much easier to use.

```
J-Run compiled Dec 13 2023 17:09:30
(c) 2019-2019 SEGGER Microcontroller GmbH    www.segger.com

Syntax:
  JRun [option option ...] elf-file

  Option                    Default       Description
  --usb <SerialNo>          not set       Set serial number of J-Link to connect to via USB.
  --ip <str>                not set       Set host name of J-Link to connect to via TCP/IP.
  --device <str>, --d <str> STM32F407IE   Set device name to str.
  --if SWD | JTAG           SWD           Select SWD or JTAG as target interface.
  --speed <kHz>             4000          Set interface speed to n kHz.
  --rtt                     Auto          Explicitly enable RTT.
  --nortt                   Auto          Explicitly disable RTT.
  --semihost                Auto          Explicitly enable semihosting.
  --nosemihost              Auto          Explicitly disable semihosting.
  --x str, --exit str       *STOP*        Set exit wildcard to str.
  --quit                    On            Automatically exit J-Run on application exit.
  --wait                    Off           Wait for key press on application exit.
  --2, --stderr             Off           Also send target output to stderr.
  --s, --silent             Off           Work silently.
  --v, --verbose            Off           Increase verbosity.
  --dryrun                  Off           Dry run. Parse elf-file only.
  --jlinkscriptfile <str>   not set       Set path of J-Link script file to use to str. Further info: wiki.segger.com/J-Link_script_files
```

JRun can do Semihosting and RTT output and redirects it to the console. For example:

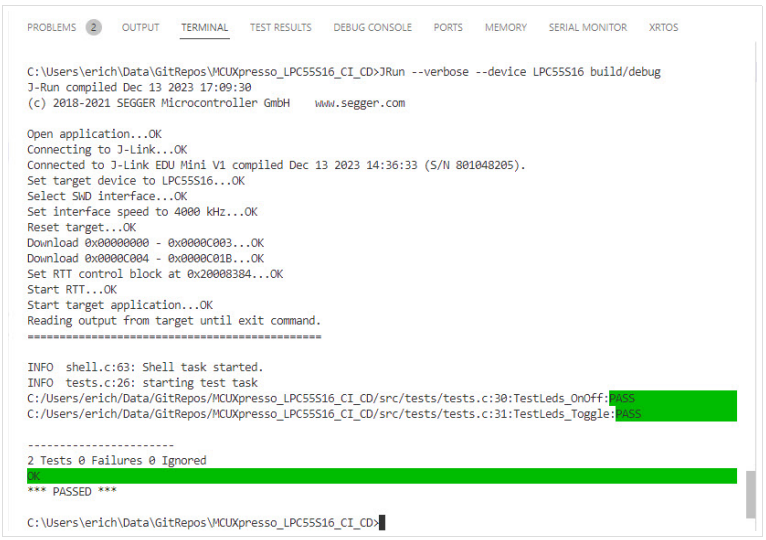
```
JRun --verbose --device LPC55S16 build/debug-test/LPC55S16_Blinky.elf
```

JRun monitors the output itself, and it can be asked to exit if it gets a special "*STOP*" string. So we can write the overall test result (passed or failed) and then exit the application:

```
1 if (nofFailures==0) {
2   McuShell_SendStr((unsigned char*)"*** PASSED ***\n", McuRTT_stdio.stdOut);
3 } else {
```

```
4     McuShell_SendStr((unsigned char*)"*** FAILED ***\n", McuRTT_stdio.stdOut);
5   }
6   McuShell_SendStr((unsigned char*)"*STOP*\n", McuRTT_stdio.stdOut); /* stop JRun */
```

Which then writes the output to the console:



— JRun Output on Console

Parameter Passing

In the above case, both tests were executed together. But sometimes it can make sense to run separated tests, with something like passing an argument to the running application. One way to do this is to pass a parameter to the application using a J-Link Script file.

The script file then looks like this which writes the value 1 at address 0x2000'0000:

```
1   int HandleAfterFlashProg(void) {
2     int res;
3     res = JLINK_MEM_WriteU32(0x20000000, 1);
4     return 0;
5   }
```

The variable at address 0x2000'0000 is allocated in a 'no-init' section (see this article):

```
1   uint32_t program_arg __attribute__((section (".uninit")));
```

The script is passed on the command-line to JRun:

```
JRun --verbose --device LPC55S16 --jlinkscriptfile src/tests/test_1.JlinkScript build/debug-test/LPC55S16_Blinky.elf
```

With this I can select and choose tests at runtime:

```
1   int test_arg = McuUnity_GetArgument(); /* get test arguments */
2   UNITY_BEGIN();
3    switch(test_arg) {
4     case 1:   RUN_TEST(TestLeds_OnOff); break;
5     case 2:   RUN_TEST(TestLeds_Toggle); break;
6     default:  RUN_TEST(TestArgFailed); break;
7   }
8   nofFailures = UNITY_END();
```

With this, I can select which test to run, or pass any other parameters to the application.

I have a feature request pending at SEGGER for JRun to have an argument for it on the command line, so I can pass a string or something else to it which then can be read by semihosting or RTT. That would make it more portable (currently I have a script creating the J-Link Script files). For example:

```
JRun --arg="test=5" .....
```

Crossing fingers that this gets implemented sometimes in the near future.

Adding Tests

Adding tests with CTest uses the following pattern:

```
add_test(
  NAME <testname>
  COMMAND <command to be executed>
)
```

For example I can add two tests like this:

```
set (JRUN_CTEST_COMMAND "JRun" --verbose --device LPC55S16 --rtt -if SWD)

add_test(
  NAME Led_1
  COMMAND ${JRUN_CTEST_COMMAND} --jlinkscriptfile "${CMAKE_CURRENT_SOURCE_DIR}/test_1.JLinkScript" ${TEST_EXECUTABLE}
)

add_test(
  NAME Led_2
  COMMAND ${JRUN_CTEST_COMMAND} --jlinkscriptfile "${CMAKE_CURRENT_SOURCE_DIR}/test_2.JLinkScript" ${TEST_EXECUTABLE}
)
```

Additionally I can set properties for the tests, like setting a timeout:

```
set_tests_properties(Led_1 Led_2 PROPERTIES TIMEOUT 15)
```

Pass or Fail

Finally, CTest needs to know if the tests were successful or not. CTest expects the test application to return 0 for no failure, and a negative return code for 'test failed'. Unfortunately, JRun always returns 0 (that's yet another pending feature request from my side). But I can tell CTest to use a Regular Expression instead on the output:

```
set (passRegex "\\*\\* PASSED \\*\\*\\*")
set (failRegex "\\*\\*\\* FAILED \\*\\*\\*")
set_property(TEST PROPERTY PASS_REGULAR_EXPRESSION "${passRegex}")
set_property(TEST PROPERTY FAIL_REGULAR_EXPRESSION "${failRegex}")

set_tests_properties(Led_1 Led_2 PROPERTIES PASS_REGULAR_EXPRESSION "${passRegex}")
set_tests_properties(Led_1 Led_2 PROPERTIES FAIL_REGULAR_EXPRESSION "${failRegex}")
```

With this, CTest properly detects failed tests. I can run CTest from the command-line:

```
ctest --extra-verbose --test-dir build/debug-test --timeout 120
```

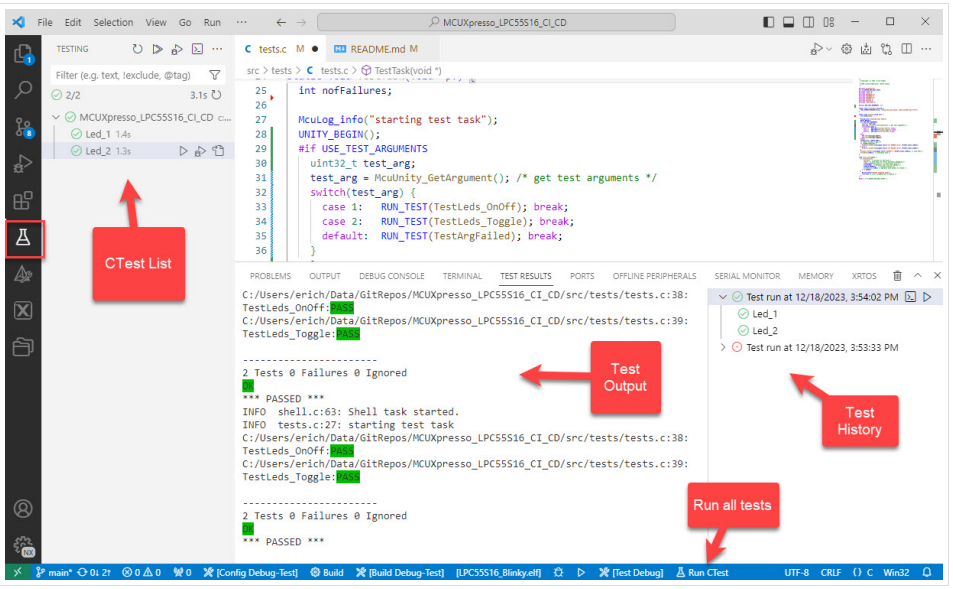
— Running CTest on the Command Line

To run a single test or a subset of tests, I can use the -R option:

```
ctest --test-dir build/debug-test -R Led_2
```

VS Code

So far, everything was with CMake, CTest and command line tools. But CTest is nicely integrated in VS Code.



— CTest with VS Code

Summary

With this, I have a complete setup and system to run on-target tests on the real hardware: from using VS Code, running CMake and CTest, configuring tests, downloading and running it on the target hardware, up to collecting test results. Clearly one does not need to test everything on the hardware, but in embedded systems there are always things which only can be tested on hardware. I hope with the presented approach and with the project and files shared on GitHub you can explore unit or system testing on hardware in a modern environment.

Happy testing 🙂

LINKS

- Project on GitHub: https://github.com/ErichStyger/MCUXpresso_LPC55S16_CI_CD
- Unity Test-Framework: http://www.throwtheswitch.org/unity
- McuLib: https://github.com/ErichStyger/McuOnEclipseLibrary
- CI/CD for Embedded with VS Code, Docker and GitHub Actions
- LoRaWAN with NXP LPC55S16 and ARM Cortex-M33
- TDD: https://en.wikipedia.org/wiki/Test-driven_development